Ezairo® Sound Designer Software Development Kit (SDK) Programmer's Guide

M-20815-031
December 2025

© SCILLC, 2025
Previous Edition © 2024
"All Rights Reserved"

Table of Contents

Page

Ezairo® Sound Designer Software Development Kit (SDK) Programmer's Guide 1

Table of Contents 2

1. Introduction 6

1.1 Purpose 6

1.2 Sound Designer Software Overview 6

1.3 Intended Audience 6

1.4 Conventions 6

1.5 Further Reading 6

2. Overview 7

2.1 Introduction 7

2.1.1 Minimum System Requirements 7

2.2 The Product Manager 7

2.3 Product Libraries 7

2.4 Accessing the Communicator Adaptor Interface 8

2.5 Manufacturing Interface 8

2.6 Sample Code 8

2.7 Asynchronous Non-Blocking Methods 8

3. Redistribution 10

3.1 Sound Designer Software Run-Time Components 10

3.2 Language Support Libraries 12

4. Language Support Libraries 13

4.1 API Concepts 13

4.1.1 Data Types 13

4.1.2 Errors 14

4.2 COM Component 17

4.2.1 Accessing the Sound Designer Software COM Component from C++ 18

4.2.2 Accessing the Sound Designer Software COM Component from C# 18

4.3 Python Language Module 18

5. Communication Interfaces 20

5.1 Getting a List of Communication Interfaces 20

5.2 Getting a Programmer 20

5.3 Using the Communication Adaptor Class 20

5.3.1 Verifying NVM Writes 20

5.3.2 Detecting a Device 21

5.3.3 Check Device 21

5.3.4 Controlling Parameter Access 21

5.3.5 Closing the Connection 21

6. Working with Libraries 22

6.1 Getting a Library 22

6.2 Loading a Library 22

6.2.1 Library Properties 22

6.2.2 Getting a List of Product Definitions 22

6.2.3 Creating a Product 22

6.2.4 Working with a Product 22

6.3 Device Initialization and Configuration 23

6.4 Firmware Updating 23

6.5 Parameter Adjustment 24

6.6 Parameter Reading and Writing 25

6.7 Graphing 26

6.8 Transducer Models 26

6.9 Using Voice Alert Data in param Files 27

6.9.1 Write Voice Alerts 27

6.9.2 Read and Verify Voice Alerts 28

6.9.3 Audition Configured Acoustic Indicators 28

6.10 Data Logging 28

6.10.1 Retrieve Data 28

6.10.2 Data Log Settings 28

6.10.3 Data Log Reset 29

6.10.4 Data Log Structure 29

6.10.4.1 Long-Term Data Log 29

6.10.4.2 Short-Term Data Log 29

6.10.4.3 Usage Time Data Log 31

6.10.5 Data Log JSON File Structure 31

6.11 Other Uses for the Product Class 32

6.11.1 Muting and Unmuting 32

6.11.2 Reading and Writing the Manufacturing Area 32

6.11.3 Automatic Receiver Detection (ARD) 32

6.11.4 Feedback Path Measurement Tool 32

6.12 Finish Working with a Device 33

7. Live Display Interface 34

7.1 Live Display Properties 34

7.2 Creating A Live Display 34

7.3 Starting a Live Display 34

7.4 Reading a Live Display 35

7.5 Stopping a Live Display 35

8. Working with a Wireless Device 36

8.1 Scanning for and Connecting to a Device 36

8.2 Handling Events from the SDK 36

8.3 Handling Disconnects 37

8.4 Controlling a Device via Bluetooth Characteristics 38

8.5 Updating MFi Information on a Device 38

8.6 Updating MFi Memory Names and Categories 38

9. Manufacturing Interface 40

9.1 Acoustic Test Box 40

9.2 Modeling 41

9.3 Calibration 41

9.4 Configure 41

CHAPTER 1

Introduction

1.1 Purpose

This manual explains how to develop software that uses the Ezairo® Sound Designer Software Application Programming Interface (API) with Ezairo Digital Signal Processing System (DSP)-based hybrids, and how to redistribute this software based on the Ezairo Sound Designer Software. It provides information to create manufacturing and fitting software, including how to develop fitting software using the Sound Designer API, and tips on how to minimize programming time.

For a description of every class, method and argument, see the *Ezairo Sound Designer Application Programming Interface Reference*.

1.2 Sound Designer Software Overview

Sound Designer Software is an integrated software package that allows hearing aid manufacturers to design, test and deploy hearing aids. The two ways of interacting with this software are through:

- A graphical user interface presented as the Sound Designer Application
- The Ezairo Sound Designer Software Development Kit (Sound Designer SDK)

The Application Programming Interface (API) is the middle layer between the user interface provided by the Sound Designer Application, and the Communications Toolkit (CTK). The API is part of the Sound Designer SDK. With the help of example applications in the Sound Designer SDK, hearing aid manufacturers can develop fitting software and manufacturing software to support their products.

This product was developed in compliance with IEC 62304 *Medical device software – Software life cycle processes*.

1.3 Intended Audience

This manual is for software developers who are familiar with object-oriented programming.

1.4 Conventions

The following conventions are used in this manual to signify particular types of information:

`monospace font`
: Class, method, argument and object names.

1.5 Further Reading

For more information, refer to the following documents:

- *Ezairo Sound Designer Application Programming Interface Reference*
- *Ezairo Sound Designer Software User's Guide*

CHAPTER 2

Overview

Sound Designer Software is intended to significantly shorten the software development time required to add new hearing instruments to a fitting module or standalone fitting software. It simplifies or eliminates the repetitive tasks of programming different types of hearing aids. This allows more time for the fitting software development team to focus on innovative user interfaces, the audiologist/patient experience, and fitting methodologies. User-facing fitting software can be built on top of the Sound Designer API.

2.1 Introduction

Sound Designer Software can be used with or without a connected hearing aid device. Without a connected device, Sound Designer Software can still be used to design products, tailor their parameters, and view a model of the resulting device response.

With a connected device, Sound Designer Software can be used to test designed products and observe their behavior. Depending on the connected device, live display options are available to view specific internal data, such as level detectors, which are streamed back in real-time.

Sound Designer Software provides interfaces that can be used to calibrate and configure hearing aids at the manufacturing level. Calibration allows for the compensation of gain mismatches due to process variability of microphones, receivers, and telecoils.

2.1.1 Minimum System Requirements

Depending on your operating system, you need the following minimum requirements for Sound Designer to function properly:

- iOS: 11 - 15+
- Android: 10 - 12+
- Windows: 10+

2.2 The Product Manager

The Product Manager class is the entry point to the Sound Designer API. Through it, instances of all other required classes are obtained.

2.3 Product Libraries

Product libraries are collections of products. Products can be hybrids that input and output electrical signals, or complete hearing aids that include shell, tubing and transducer characteristics. Product libraries handle the parameter mapping and generating data for characteristic curves. A product helps to plot characteristic curves by calculating the output for given input levels, frequencies, and product parameter settings.

As a first approximation, the manufacturer can combine an electrical device model with microphone and receiver models (which would correspond to the acoustic input model and acoustic output model, respectively). The manufacturer can then take several measurements of actual assembled hearing instruments to refine the acoustic models or match the device response against the golden reference. A key goal of Sound Designer Software is to assist manufacturers in the rapid development of new products by removing the requirement to measure acoustic curves for every parameter combination.

Manufacturers might want to combine products into libraries by product line, or combine their entire product family into a single library. These capabilities allows manufacturers to create a single library for one hybrid with multiple products that account for various acoustic setups (different transducer models, shells, and tubing) in their product line.

Details of working with libraries and products can be found in Chapter 6 "Working with Libraries" on page 22.

2.4 Accessing the Communicator Adaptor Interface

Communication Adaptors are used to connect products to devices. Details can be found in Chapter 5 "Communication Interfaces" on page 20.

2.5 Manufacturing Interface

The Manufacturing Interface contains all methods and classes required to model and calibrate a device. This interface grants access to supported acoustic test boxes, allows levels and gain settings to be read back from a hearing aid, and also lets a manufacturer write compensation gains for the purposes of acoustic calibration.

For example, a manufacturer can generate stimulus tones for a hearing aid in an acoustic test box, such as a Frye FONIX 8000, and also read back current level detector values from the device for the purposes of building an acoustic model of the device microphone. Conversely, a manufacturer can instruct a hearing aid device to generate a series of tones, and have an acoustic test box measure its response to model the device receiver.

Any calculated compensation gains are stored to the product via the Manufacturing Interface and are written to the device when configured. For more information, see Chapter 9 "Manufacturing Interface" on page 40.

2.6 Sample Code

The Sound Designer SDK provides two sample code applications which both exercise the API and provide examples of using the API:

COMTests
: This application consists of a suite of automated tests, and accesses the Sound Designer SDK via COM. Each test serves as a small example of how to use the API to perform standard tasks.

ParamSample
: This application uses the Sound Designer SDK to implement a simple fitting application. It allows a user to select a product and communicate with it. Users can read and write individual parameters to the device.

WARNING: Unintended usage of Sound Designer or its sample code can result in extremely high gain leading to temporary threshold shift and/or ringing in your ears. Do not place a hearing aid into your ear before sanity-testing it with lab equipment.

2.7 Asynchronous Non-Blocking Methods

In some cases, executing a method will take a noticeable amount of time to complete. Configuring a device for example, can take several seconds. In this scenario, the execution of the `ConfigureDevice` method holds until it completes. You might prefer to use one of the provided non-blocking methods instead.

The use of the asynchronous non-blocking methods is similar throughout the API. Methods that do not return any data have `Begin` prefixed to the name of the method. Methods that do return data also contain a companion method that is prefixed with `End`. Below is an example of the non-blocking `InitializeDevice` method which returns a boolean value indicating whether a device has been initialized. The following methods are used:

- `AsyncResult Product::BeginInitializeDevice(CommunicationAdaptor adaptor)`
- `bool Product::EndInitializeDevice(AsyncResult async)`

The `AsyncResult` class is used to monitor the status of the executing non-blocking method. In the future it will be used to query the progress of the method. Presently, it can be used to determine whether the call has finished. It is also used as a container to hold the result of the method, if there is one.

The `bool AsyncResult::IsFinished()` method can be polled to determine when the non-blocking method has completed its execution.

After determining that the non-blocking method is complete, you can call a corresponding `End` prefixed method to decode and return the expected value. In this case, it is a true or false `bool`, as indicated in the `EndInitializeDevice` example above.

CHAPTER 3

Redistribution

This chapter gives an overview of the binaries included in the Sound Designer SDK, and where they need to reside in your software.

3.1 Sound Designer Software Run-Time Components

The Sound Designer Software run-time components are normally installed in the directory *\SoundDesignerSDK\bin* on the Windows® platform. Depending on which software components you want to use, the libraries needed to use the SDK with a library created with the library manager are as follows.

Core Libraries (always required):

- *FryeCom.dll*: For interfacing with the Frye FONIX sound chamber.
- *noahlinklib.dll*: Implements support for Noahlink wireless.
- *rsl10lib.dll*
- *sd.config*: A configuration file that is required to support the different communication adaptor interfaces.
- *SoundDesigner.Archive.dat*
- *SoundDesigner.Core.dll*: Implements the `ProductManager` class and is the entry point to the Sound Designer API.
- *SoundDesigner.Framework.dll*
- *SoundDesigner.Modeling.dll*
- *SoundDesigner.Modules.dll*
- *SoundDesigner.Manufacturing.dll*

The *SoundDesigner.*.dll* files implement specific features and interfaces provided by Sound Designer Software. These files are detected and loaded automatically by the Sound Designer Software Product Manager when you first access Sound Designer Software in your program.

HIMSA Noahlink Wireless DLL files

The Noahlink Wireless "Nibelung" Driver files can be found in the Noahlink Wireless SDK (HIMSA) and we do not redistribute them.

To use the Noahlink Wireless programmer in the SDK, set the `ProductManager.BLEDriverPath` property with the location of the driver files before calling `ScanForWirelessDevices` or `CreateWirelessCommunicationInterface`. In Sound Designer, set the path after selecting the programmer in General Settings. All required DLLs need to be in a flat list in that directory.

The C# Sample Code has also been updated (in *Connection.cs*) to show the setting of the `BLEDriverPath` property. To run the C# sample code, make changes in *Constants.cs* so that the Programmer is Noahlink, `IsProgrammerWireless` is true, and the `WirelessDeviceName` and the `DriverLocation` are the local Noahlink driver path.

See the *README* file in the C# Sample code directory for troubleshooting.

This implementation was tested with Noahlink Wireless SDK 2.0.5 DLLs supporting both the first and second generation of the Noahlink Wireless.

RSL10 Dongle

Using an onsemi RSL10 Dongle requires its driver and the RSL10 SDK DLLs provided in the Sound Designer SDK. The SDK DLLs are with the sample code in the *samples/win/bin/rsl10/* folder. Place it beside the Core libraries. The folder must keep the name *rsl10*. The contents of the folder are:

- OnSemi.RSL10.Driver.dll
- OnSemi.RSL10.Adapter.dll
- OnSemi.RSL10.Adapter.dll.config
- Arendi.BleLibrary.dll
- Arendi.DotNETLibrary.dll
- Arendi.DotNETLibrary.Windows.dll
- log4net.dll
- Newtonsoft.Json.dll

To use the RSL10 Dongle in the SDK, call `ScanForWirelessDevices` or `CreateWirelessCommunicationInterface` with the COM port as listed in the Control Panel's **Device Manager** > **Ports (COM & LPT)**. In Sound Designer, set the COM port after selecting the programmer in **General Settings**.

If you are using C# to interface to the SDK:

- *SDNative.dll*
- *SDCS.dll*

If you are using COM to interface to the SDK:

- *SDCOM.dll*
- *SDCOM.dll.manifest*

If you need to interface to a Frye FONIX sound chamber:

SoundDesigner.TestBox.FONIX.dll

Other package dependencies for non-wireless connectivity:

Besides the DLL files above, you have to install the Communication Toolkit (CTK) package by launching the CTK installer from your installation software. The redistributable package is included with the Sound Designer Software in *C:\Program Files (x86)\ON Semiconductor\Sound Designer\SD\redistribution\CTK*. You can choose the *.msi* or the *.msm* file. If you choose the latter, and you are using a WiX installer, you can install the CTK as follows:

1. Add the file below to the WiX project which references the merge module:

```
 <Wix xmlns="http://schemas.microsoft.com/wix/2006/wi">
 <Fragment>
 <DirectoryRef Id="INSTALLDIR">
 <Merge Id="CTKRuntime" Language="1033"
SourceFile="$(var.SolutionDir)..\redistribution\ctk\CTKRuntime.msm" DiskId="1" />
 </DirectoryRef>
 <Feature
 Id="CTKRuntime"
 Title="CTK Runtime"
 Level="1">
 <MergeRef Id="CTKRuntime"/>
 </Feature>
 </Fragment>
 </Wix>
```

2. Reference the feature in the file with `<FeatureRef Id="CTKRuntime"/>`.

3.2 Language Support Libraries

You access the Sound Designer API through a language-specific library. These libraries are described in Chapter 4 "Language Support Libraries" on page 13. With the exception of the Sound Designer Software COM component, you must install the appropriate language-specific library in a location where your program can access it. The Sound Designer Software COM component is installed with the other Sound Designer Software run-time components in the file *SDCOM.DLL*. If you are using the COM component, register *SDCOM.dll* on the Windows system.

If you are using C# to interface to the SDK, install *SDNative.dll* and *SDCS.dll*.

CHAPTER 4

Language Support Libraries

The Sound Designer API can be accessed from any language that supports Microsoft COM. This chapter provides examples for C++ and C#. The API can also be accessed from the Python scripting language.

4.1 API Concepts

The Sound Designer API is divided into several *interfaces*. Each interface contains members that are divided into two categories: *methods* which take parameters, perform operations, and return results; and *properties* which are named values. Some properties are *read-only*; others are *read/write*.

For some interfaces, you can directly create an *object* that implements that interface. For example, you can create an object that implements the `CommunicationAdaptor` interface. Objects that implement other interfaces are obtained by calling methods or reading properties on objects.

Some languages have special support for lists. Interfaces that behave like lists (e.g., `TransducerModelList` or `ProductDefinitionList`) have a `Length` property and a method named `GetItem()` that is designated as an *indexer*. Where support exists in the programming language, these list interfaces can be accessed using the array subscript operator.

4.1.1 Data Types

In addition to interfaces, the Sound Designer API uses a number of basic data types. These data types are represented by the corresponding native data type in each language. The basic data types are:

- `string`
- `bool`
- `int` (32-bit signed integer)
- `byte` (8-bit unsigned integer)
- `short` (16-bit unsigned integer)

In languages that do not support unsigned integers, the corresponding signed native data type is used.

The Sound Designer API also uses two special data types that are actually 32-bit integers:

`error`
: a 32-bit integer error identifier.

Bit(s)	Description
31	Severity • 0 = info • 1 = error
16-30	Category
0-15	Error Code

Error codes are discussed more in Section 4.1.2 “Errors” on page 14.

`version`
: a 32-bit integer version identifier.

Bits	Description
24-31	Major Version
16-23	Minor Version
0-15	Revision

Version numbers can easily be compared using the < and > operators. Changes to the revision number indicate bug fixes. Changes to the minor version number indicate backward-compatible changes. Changes to the major version number indicate changes that are not backward compatible.

4.1.2 Errors

Every Sound Designer Software operation (creating an object, calling a method, and getting or setting a property) can generate an error. Sound Designer Software errors have three representations: a 32-bit integer, a short text identifier, and a detailed text description. Errors are returned to the caller using each programming language's exception mechanism. The `ProductManager` class contains three methods for converting among these error types:

`const char* ProductManager::GetErrorString(int errorCode)`
Converts a 32-bit integer error identifier to a short text identifier.

`const char* ProductManager::GetDetailedErrorString(int errorCode)`
Converts a 32-bit integer error identifier to a short, English text description.

`const char* ProductManager::GetDetailedErrorStringFromName(const char* errorName)`
Converts a short text error identifier to a short, English text description.

`int ProductManager::GetErrorCodeFromName(const char* errorName)`
Converts a short text error identifier to a 32-bit integer error identifier.

The Sound Designer Software errors are listed in the table "Error Codes" (Table 1). In some languages, certain errors are translated into native error types.

Table 1. Error Codes

Short Text Identifier	Integer Identifier	Description
S_OK	0x00000000	No Error.
E_OUT_OF_MEMORY	0x80000001	There is not enough memory to complete the operation.
E_INDEX_OUT_OF_BOUNDS	0x80000100	The index specified is out of bounds.
E_UNINITIALIZED_OBJECT	0x80000101	The object is not initialized.
E_UNKNOWN_NAME	0x80000102	The name specified is not recognized.
E_NOT_IMPLEMENTED	0x80000103	This function is not implemented.
E_INVALID_OBJECT	0x80000104	The object is invalid.
E_PARSE_ERROR	0x80000105	An error occurred parsing the input.
E_INVALID_SETTING	0x80000106	A setting is invalid.
E_INVALID_STATE	0x80000108	The system is in an invalid state for this operation.

Table 1. Error Codes (Continued)

Short Text Identifier	Integer Identifier	Description
E_NOT_FOUND	0x8000010B	The entry could not be found.
E_CHECKSUM_FAILURE	0x8000010D	A checksum failed.
E_INVALID_VERSION	0x8000010F	An incompatible hardware version was found.
E_GENERAL_TIMEOUT	0x80000110	A timeout occurred.
E_INTERNAL_ERROR	0x80000111	An unexpected internal error occurred.
E_UNKNOWN_ID	0x80000112	The id specified is not recognized.
E_CANCELLED	0x80000113	An asynchronous operation was canceled.
E_INCORRECT_KEY	0x80000114	An incorrect key was specified.
E_INVALID_OPERATION	0x80000115	An invalid operation was attempted.
E_INVALID_ARGUMENT	0x80000116	An invalid argument was specified.
E_CRC_FAILURE	0x80000117	A CRC check failed when writing to EEPROM.
E_IN_MODELING_MODE	0x80000118	This function cannot be used when in modeling mode. Exit this mode before proceeding.
E_NOT_IN_MODELING_MODE	0x80000119	This function cannot be used when not in modeling mode. Enter this mode before proceeding.
E_EEPROM_ERROR	0x80000120	An error writing to EEPROM occurred.
E_FILE_NOT_FOUND	0x80010001	The file was not found.
E_END_OF_FILE	0x80010002	The end of the file was encountered.
E_BAD_DATA_FORMAT	0x80010003	The data file was not formatted correctly.
E_INVALID_FILE_VALUE	0x80010004	A data value in the file was incorrect.
E_CANNOT_OPEN_FILE	0x80010005	Cannot open output file.
E_UNKNOWN_ERROR	0x8001FFFF	An unknown error occurred.
E_INVALID_MODULE	0x80020001	The module is invalid.
E_MODULE_API_VERSION	0x80020002	The module is out of date.
E_CANNOT_LOAD_CTK	0x80020003	Cannot load CTKManager.
E_CONFIG_ERROR	0x80020004	sd.config could not be found or was invalid.
E_CANNOT_OPEN_DRIVER	0x80030001	Cannot open the communication hardware driver. The driver might already be open, or the drivers are not installed correctly.
E_CANNOT_CONFIG_DRIVER	0x80030002	Cannot configure the communication hardware or driver. Check that the hardware and drivers are installed correctly.
E_UNEXPECTED_CTK_ERROR	0x80030003	An unexpected CTK error occurred.
E_UNSUPPORTED_INTERFACE	0x80030004	The selected communication interface is not supported.
E_UNSUPPORTED_PROTOCOL	0x80030005	The selected protocol is not supported with the current communication interface.
E_WIRELESS_ERROR	0x80030006	This API call is not supported over a wireless connection.

Table 1. Error Codes (Continued)

Short Text Identifier	Integer Identifier	Description
E_SEND_FAILURE	0x80040001	An error occurred sending data. Check your communication hardware.
E_RECEIVE_FAILURE	0x80040002	An error occurred receiving data. Check your communication hardware.
E_RECEIVE_TIMEOUT	0x80040003	A timeout occurred receiving data. Check your communication hardware.
E_NOT_DETECTED	0x80040004	No device was detected.
E_VERIFICATION_ERROR	0x80040005	A write to non-volatile memory failed to match during verification.
E_NEW_DEVICE	0x80040006	A new device was detected. It must be identified prior to further communication.
E_CRC_ERROR	0x80040007	A CRC error was detected.
E_INCOMPATIBLE_DEVICE	0x80040008	The device is not compatible with the current product.
E_UPGRADE_FAILED	0x80040009	The update completed; however, the new firmware identifier and version number do not match the expected values.
E_UNRECOGNIZED_DEVICE	0x80040010	The connected device was not recognized.
E_UNCONFIGURED_DEVICE	0x80040011	The device must be configured before its parameters may be read or written.
E_SIMULATOR_ERROR	0x80040012	A simulator error occurred.
E_ALREADY_OPEN	0x80040013	A connection to the device is already open.
E_NOT_READY	0x80040014	The acoustic test box is not ready.
E_COMMAND_FAILURE	0x80040015	The acoustic test box was not able to process the command sent.
E_NO_RESPONSE	0x80040016	No response from the acoustic test box.
E_BAD_RESPONSE	0x80040017	Bad response from the acoustic test box.
E_NOT_OPEN	0x80040018	The connection to the acoustic test box is not open.
E_INVALID_FIRMWARE	0x80040019	The device is in an invalid state. Please call use UpdateDevice to restore the device to a working state.
E_ACK_ERROR	0x80040020	An invalid acknowledgment was received while attempting to update the radio.
E_NOT_INITIALIZED	0x80040021	Product not initialized. Call InitializeDevice before proceeding.
E_PARAMETER	0x80050001	A parameter value is invalid.
E_VALUE_OUT_OF_BOUNDS	0x80050002	A parameter value specified is out of range.
E_PARAMETER_NOT_FOUND	0x80050003	Parameter not found.
E_PARAMETERS_LOCKED	0x80050004	Parameter access is locked. Unlock using the correct key first.
E_PARAM_CALC_ERROR	0x80050005	An internal error with a parameter calculation has occurred.
E_NO_PRODUCT_SET	0x80050006	No Product set. Call SetProduct before proceeding.

4.2 COM Component

The Sound Designer Software COM component provides access to the Sound Designer API from any language that supports COM. This includes C++, C# and other .NET languages, and many other languages. For more information, consult the documentation included with your programming language tools.

The COM component is implemented in one file:

SDCOM.DLL
: The COM component.

This file also contains the COM type library that some programming languages need to access the COM component. You can also access the COM component using its ProgIDs: `SD.ProductManager.1` or `SD.ProductManager`.

Sound Designer Software interfaces are implemented as COM interfaces in the type library. The COM interfaces are the names of the Sound Designer Software interfaces prefixed with `I`, following the COM naming conventions. For example, the `TransducerModelList` Sound Designer Software interface is exposed as `ITransducerModelList` in the COM type library.

Sound Designer Software methods are implemented as COM methods. The COM methods have the same name as the Sound Designer Software methods.

Sound Designer Software properties are implemented as COM "propget" and "propput" methods. Some programming languages present these as properties, while others require you to use accessor methods.

If an object can be created for a Sound Designer Software interface, then there is a COM co-class with the name of the Sound Designer Software interface prefixed with `C`, following the COM naming conventions. For example, you can create an instance of `CProductManager` which implements the `IProductManager` interface.

For interfaces that behave like lists, the indexer is replaced with a "propget" method named `Item`. Many programming languages allow you to use the array subscript operator to access this method.

The parameters passed through the COM component methods have been selected to maximize programming language compatibility. Arrays are passed as `SAFEARRAY`s and strings are passed as `BSTR`s. The caller must allocate and free arrays that are passed into methods. The Sound Designer Software COM component allocates arrays that are returned from methods and the caller must free them. Where methods are overloaded to accept more than one parameter type (e.g., some `GetItem()` methods can accept a string or an integer), the actual parameter type is `VARIANT`. Consult the documentation included with your programming language tools for more information on these types and how your programming language handles them.

All Sound Designer Software methods can raise exceptions through the COM `ErrorInfo` mechanism. Some programming languages use the `ErrorInfo` mechanism to generate native exceptions or errors. In other programming languages, you must use the Win32 `GetErrorInfo()` function to retrieve the error information. The `Description` property of the `ErrorInfo` object contains the short text identifier of the error. You can obtain the short, English text description of the error using the `ProductManager.GetDetailedErrorStringFromName()` method. The following errors are converted into native `HRESULT` errors instead of `ErrorInfo` errors:

- `E_OUT_OF_MEMORY` is converted to `E_OUTOFMEMORY`.
- `E_NOT_IMPLEMENTED` is converted to `E_NOTIMPL`.

4.2.1 Accessing the Sound Designer Software COM Component from C++

To access the Sound Designer Software COM Component from C++, import the Sound Designer Software type library using `#import SDCOM.DLL`. There are two ways to call COM methods from C++: raw interfaces, and error handling wrapper functions. Consult the MSDN documentation for more information.

C++ provides conversion routines to generate `BSTR`s for strings, and `VARIANT`s where necessary. Helper classes exist to handle `SAFEARRAY`s.

When calling an overloaded Sound Designer Software method that requires a `VARIANT` parameter, you can prepare and pass a `VARIANT` data structure, or you can take advantage of automatic conversions in C++ and pass a `string` or a `long` integer directly. To pass an `int` data type, you must first cast it to a `long` data type.

4.2.2 Accessing the Sound Designer Software COM Component from C#

To access the Sound Designer Software COM Component from any .NET programming language, use the *tlbimp.exe* tool to extract the type library from *SDCOM.DLL* and generate a Runtime Callable Wrapper (RCW) for the COM library. Specify the output filename and namespace for the RCW assembly when you run *tlbimp.exe*.

Consult the Microsoft .NET Framework SDK documentation for more information about accessing COM objects from .NET languages.

All basic data types are converted transparently between .NET data types and COM data types. Arrays require some special treatment:

- Specify the `ref` keyword to pass arrays to methods by reference.
- Specify the `out` keyword to mark arrays that are returned by methods as output parameters. It is sufficient to declare the array variable; you do not need to allocate the array before calling the Sound Designer Software COM component.

For example:

```
int length = 30;
byte[] data = new byte[30];
Product.WriteManufacturerData(0, length, out data);
```

In .NET languages, all Sound Designer Software methods can throw `COMException`. The `Message` property of this object contains the short text identifier of the error.

4.3 Python Language Module

The Sound Designer Software Python language support library is implemented in one file. Use the appropriate file for your Python version:

python/2.7/sd.pyd
: The native Python 2.7 module. If you are using Python 2.7, place this file in your Python 2.7 search path.

Sound Designer Software interfaces are implemented as Python classes in the `sd` module. The Python classes have the same name as the Sound Designer Software interfaces.

Sound Designer Software methods are implemented as Python methods. The Python methods have the same name as the Sound Designer Software methods.

Sound Designer Software properties are implemented as Python attributes. If a property is read-only, attempting to set the attribute produces a `TypeError`.

If an object can be created for a Sound Designer Software interface, the Python class has a constructor that takes no parameters. For example, you can create an instance of the Python `SD.ProductManager` class.

Python has special support for list operations. To get the length of a list object, you can read the `Length` attribute or you can use the built-in Python `len()` function. They produce identical results; which to use is a matter of preference. To retrieve an item in a list object, use the array subscript operator (`[]`), which calls the Sound Designer Software `GetItem()` method. Other array operations, including array slices and negative indices, are not supported. The `GetItem()` method is not available in Python.

When calling Sound Designer Software methods that use arrays, pre-allocate all Python arrays with an appropriate length before calling the method. For example:

```
length = 30
data = [30]
Product.WriteManufacturerData(0, length, data)
```

All Sound Designer Software Python methods and attribute accesses can raise exceptions. In Python, the `sys.exc_info()[1]` contains the short text identifier of the error. You can obtain the short, English text description of the error using the `ProductManager.GetDetailedErrorStringFromName()` method. The following errors are converted into native error types:

- `E_OUT_OF_MEMORY` is converted to `MemoryError`.
- `E_UNKNOWN_NAME` is converted to `KeyError`.
- `E_NOT_IMPLEMENTED` is converted to `NotImplementedError`.

CHAPTER 5

Communication Interfaces

This chapter describes how to get a list of the communication interfaces and protocols that the Sound Designer API supports, and how to use a programming box, also known as a *programmer*.

5.1 Getting a List of Communication Interfaces

Under the Product Manager, use these methods to obtain a list of supported communication interfaces:

- `int ProductManager::GetCommunicationInterfaceCount()`
- `const char* GetCommunicationInterfaceString(int index)`

The `GetCommunicationInterfaceCount` method returns the number of supported communication interfaces minus one. It is used as an index that is passed into the `GetCommunicationInterfaceString` method to obtain the communication interface name. A complete list of supported communication interfaces can be obtained by calling `GetCommunicationInterfaceString` for all valid indices. An example of typical interfaces that are supported would be the HI-PRO, DSP programmer version 3.0, and the Binho Pulsar USB Host Adapter.

5.2 Getting a Programmer

The communication adaptor class handles all communication in Sound Designer Software. The following method can be used to instantiate a programmer for any of the supported communication interfaces:

```
CommunicationAdaptor ProductManager::CreateCommunicationInterface(const char* interface_
name, CommunicationPort port, const char* interface_options)
```

interface_name
: This is the name of the communication interface obtained by calling `GetCommunicationInterfacesString`. Refer to Section 5.1 “Getting a List of Communication Interfaces” on page 20.

port
: The port specifies whether to use the right or left port on the programmer, if supported.

interface_options
: This argument can be used to override the default CTK interface settings. Generally, you should pass in an empty string (“ ”) to use the defaults.

5.3 Using the Communication Adaptor Class

This section describes how to use a communication adaptor to verify writing to non-volatile memory (NVM), detect a device, check a device, and close the connection.

5.3.1 Verifying NVM Writes

The communication adaptor class can be configured to verify all writes to NVM. When this setting is enabled, every write to NVM is read back and compared. If the comparison fails, an error is thrown (for more information about specific errors, see Section 4.1.2 “Errors” on page 14). The `VerifyNvmWrites` property is used to enable or disable the setting:

```
bool CommunicationAdaptor.VerifyNvmWrites
```

NOTE: When this setting is enabled, programming time will be significantly increased.

5.3.2 Detecting a Device

The `CommunicationAdaptor::DetectDevice()` method returns a `DeviceInfo` class, which can be used to read various device-specific IDs. If no device is detected, an error is thrown and the `DeviceInfo` that is returned is NULL.

To call `DetectDevice` asynchronously, the communication adaptor class also provides `AsyncResult CommunicationAdaptor::BeginDetectDevice()` and `DeviceInfo CommunicationAdaptor::EndDetectDevice(AsyncResult async)` methods. See Section 2.7 "Asynchronous Non-Blocking Methods" on page 8 for more information.

5.3.3 Check Device

The `CheckDevice` method is used to determine whether the connected device has changed since the last time `DetectDevice` was called. If a different device is now connected to the programmer since the last `DetectDevice` was performed, this method returns `false`:

```
bool CommunicationAdaptor::CheckDevice()
```

5.3.4 Controlling Parameter Access

If a device contains a configured parameter lock key, parameter access on the device is restricted. The parameter lock state of a device can be determined from the `DeviceInfo.ParameterLockState` property as described in Section 5.3.2 "Detecting a Device" on page 21. The `CommunicationAdaptor` interface contains two methods used to lock or unlock parameter access. `CommunicationAdaptor::UnlockParameterAccess(int key0, int key1, int key2, int key3)` can be used to unrestrict parameter access. The key is four 24-bit words long. This key can be set as described in Section 9.4 "Configure" on page 41. If the unlock was successful, performing a `CommunicationAdaptor::DetectDevice()` call again returns a `DeviceInfo` with the `ParameterLockState` property set to `false`. The parameter access can again be restricted by calling the `CommunicationAdaptor::LockParameterAccess()` method.

5.3.5 Closing the Connection

Once a communication adaptor instantiation is no longer needed, sever its connection to the device by calling the `Close` method:

```
void CommunicationAdaptor::Close()
```

Depending on the programmer used, failure to call `Close` can result in being unable to open a new programmer connection.

CHAPTER 6

Working with Libraries

In the Sound Designer Software, a library (also called a *Product Library*) is a collection of products. Products can be Systems-in-Packages (SiPs) that input and output electrical signals, or hearing aids complete with shells, tubing and transducers that input and output acoustic signals. From a library, we gain access to the product and transducer classes.

6.1 Getting a Library

To load a library from a file, call the following method:

```
Library ProductManager::LoadLibraryFromFile(const char* path)
```

where `path` is the path to the library file (with extension *.library*).

6.2 Loading a Library

This section is about how to use a product manager to load a library from a file. You will use the resulting library object to get a list of products within the library.

6.2.1 Library Properties

Various properties are associated with a given library:

- Library ID which is a unique identifier for the library
- Custom library description
- Manufacturer ID
- Platform name (i.e., the onsemi chip that will be used in your product)

All these IDs are set when the library is created using the Library Manager in Sound Designer Software.

6.2.2 Getting a List of Product Definitions

The `Library` class provides a property called `Library.Products` which is of type `ProductDefinitionList`. Each `ProductDefinition` in the list describes a given product and provides properties to obtain various product specific IDs and a method to create the product itself. The `ProductDefinition` class can also be used to check if a compatible firmware update is available and apply it if needed.

6.2.3 Creating a Product

To get a `Product` class, call the following method:

```
Product ProductDefinition::CreateProduct()
```

6.2.4 Working with a Product

All interactions with a product occur through the general instance of the `Product` class. Through it, you can initialize access to a device, access a list of parameter memories, read and write those parameters to a device, and configure a device. Additional methods to read and write manufacturing data, reset and mute the device are also available.

The typical procedure for working with a `Product` is:

1. Get `ParameterMemory` objects for memory-specific and system parameters.
2. Get a `ParameterList` object from each `ParameterMemory`.
3. Get specific `Parameter` objects from each `ParameterList`. You can look up `Parameter` objects within a `ParameterList` by name. Read or write to this parameter.

4. Read or write from a device by calling `Product::ReadParameters` or `Product::WriteParameters`.
5. Configure a device by calling `Product::ConfigureDevice`.
6. Close the connection to the device using `Product::CloseDevice`.

6.3 Device Initialization and Configuration

To initialize communication with a device, the `Product` class provides the `Product::InitializeDevice (CommunicationAdaptor adaptor)` method. Create the `CommunicationAdaptor` object as described in Section 5.2 "Getting a Programmer" on page 20. The method returns True if the product was successfully initialized with the device, or False otherwise. A product successfully initializes with a device only if the device is running a compatible firmware load and has been previously configured with said product. Many of the `Product` methods are not functional before a device is configured.

To configure a device with a product, use the `Product::ConfigureDevice()` method. To call `ConfigureDevice` asynchronously, use the `AsyncResult Product::BeginConfigureDevice()` method from the `Product` class. See Section 2.7 "Asynchronous Non-Blocking Methods" on page 8 for more information.

6.4 Firmware Updating

When a new version of firmware is released for a product, the Sound Designer SDK is capable of updating the firmware when connected to a compatible device. This feature is supported when connected via wired and wireless programmers. Using a wired connection, the SDK can update both the DSP and RSL10 radio firmware. Using a wireless connection, the SDK can only update the RSL10 radio firmware. The `ProductDefinition` class provides methods to check for compatibility and perform the update if desired.

The Firmware ID and version number of the update can be determined with the `const char* ProductDefinition.GetupdateFirmwareId` and `const char* ProductDefinition.GetupdateFirmwareVersion` properties. If the connection is wireless, use the `const char* ProductDefinition.GetUpdateRadioApplicationVersion` property to get the radio application version.

To check if the firmware update is compatible with the connected device, use the `DeviceCompatibility ProductDefinition::GetDeviceCompatibility(CommunicationAdaptor adaptor)` method. The `CommunicationAdaptor` object needs to be created, as described in Section 5.2 "Getting a Programmer" on page 20. The table "Device Compatibility" (Table 2) describes each of the device compatibility states.

Table 2. Device Compatibility

DeviceCompatibility	Description
kCompatibleUpToDate	The detected device is running up-to-date firmware.
kCompatibleNewer	The detected device is running a compatible firmware that is newer than the version Sound Designer Software supports. Updating the device would result in a firmware downgrade.
kCompatibleOlder	The detected device is running a compatible firmware that is older than the version Sound Designer Software supports. It is possible to upgrade this device.
kCompatibleRecover	The detected device is in an invalid state. Performing an update attempts to restore it.
kIncompatible	No compatible update is available for the connected device.

If a compatible update is available, use the `ProductDefinition::UpdateDevice(CommunicationAdaptor adaptor)` method. To call `UpdateDevice` asynchronously, you can use the `AsyncResult ProductDefinition::BeginUpdateDevice(CommunicationAdaptor adaptor)` method in the `ProductDefinition` class. See Section 2.7 "Asynchronous Non-Blocking Methods" on page 8 for more information.

If the `DeviceCompatibility` state is returned as `kCompatibleRecover`, the SDK's access to the device is limited. A device can report this state if a previous firmware update has failed. `CommunicationAdaptor::DetectDevice()` to return device info is the only function that can be called until `ProductDefinition::UpdateDevice(CommunicationAdaptor adaptor)` is called to attempt a firmware recovery. If there has been a previous wireless firmware update failure, the Firmware ID returned by `DetectDevice` is likely to be `E7160SL FOTA`.

6.5 Parameter Adjustment

All product parameters are contained within an instance of a `ParameterMemory` class. There is one `ParameterMemory` object per memory for a product, plus one additional `ParameterMemory` for system parameters. For example, a product with four memories will have a total of five `ParameterMemory` objects, accessible via the `ParameterMemoryList Product.Memories` property.

Each `ParameterMemory` contains a list of parameters in a `ParameterList`. This `ParameterList` can be obtained using the `ParameterList ParameterMemory.Parameters` property. Within a `ParameterList`, you can obtain an individual `Parameter` by calling `Parameter ParameterList::GetById(const char* id)` where the `id` is the string name for a given parameter. For example, to access the channel 1 lower threshold parameter from the WDRC, you can call `ParameterList::GetById("X_WDRC_LowLevelThreshold[0]")`.

A `Parameter` can be one of several types defined by the `ParameterType` enumeration. Depending on the type, the properties used to get and set values for a parameter differ. (See the table "Parameter Value Access Methods According to Type" (Table 3).

Table 3. Parameter Value Access Methods According to Type

ParameterType	Access Properties
kInteger	• int Parameter.Value • int Parameter.Min • int Parameter.Max • int Parameter.DefaultValue
kByte	• int Parameter.Value • int Parameter.Min • int Parameter.Max • int Parameter.DefaultValue
kDouble	• double Parameter.DoubleValue • double Parameter.DoubleMin • double Parameter.DoubleMax • double Parameter.DoubleDefaultValue

Table 3. Parameter Value Access Methods According to Type (Continued)

ParameterType	Access Properties
kBoolean	• bool Parameter.BooleanValue • bool Parameter.BooleanDefaultValue
kIndexedList	• int Parameter.Value • int Parameter.Min • int Parameter.Max • int Parameter.DefaultValue • IndexedList Parameter.ListValues • double IndexedList[index]
kIndexedTextList	• int Parameter.Value • int Parameter.Min • int Parameter.Max • int Parameter.DefaultValue • IndexedTextList Parameter.TextListValues • const char* IndexedTextList[index]

Parameters of type `IndexedList` and `IndexedTextList` always have index values from 0 to the number of items in the list minus 1. These indices are accessed using `Parameter.Value`. To access the value associated with a given index for an `IndexedList` or `IndexedTextList`, use `Parameter.ListValues[index]` or `Parameter.TextListValues[index]` respectively.

6.6 Parameter Reading and Writing

Parameters are inherently memory-specific; each `Parameter` instance is attached to a `ParameterSpace`. The SDK maintains an internal cache of parameters. Their values can be manipulated in any memory and in any order. For instance, you might set the value of a `Parameter` residing in `kSystemActiveMemory`, and then set the value of another `Parameter` residing in `kNvmMemory0`; all interaction with parameter values takes place offline, and only affects the values in the SDK's internal cache.

However, when interacting with the device, you can only read or write the parameters within a single `ParameterSpace` memory at a time.

Once an initialized connection to a configured device is established, read parameters from and write parameters to the device using the following methods:

- `Product::ReadParameters(ParameterSpace memory)`
- `Product::WriteParameters(ParameterSpace memory)`

The `ParameterSpace` enumeration passed into the above method determines the location on the device that is read or written. For example, specifying a `ParameterSpace` of `kNvmMemory0` reads or writes to memory 0. If we want to read or write to the active memory, use `ParameterSpace kActiveMemory`. The `ParameterSpace` enumerations `kSystemNvmMemory` and `kSystemActiveMemory` can be used to read or write to System NVM or System Active Memory.

The current active memory of the device can be set using the `int Product.CurrentMemory` property. The current active memory of the device can be set using the `Product.SwitchToMemory` API call. The current active

memory of the device can also be set using the int `Product.CurrentMemory` property. However, the active memory of the device is not switched until you call either `Product::ReadParameters(ParameterSpace memory)` or `Product::WriteParameters(ParameterSpace memory)`.

6.7 Graphing

Sound Designer Software implements an acoustic model for each product which is used to generate various types of graphs. The two graphs presently supported are a frequency response graph and input/output graph.

A list of graphs that a product supports can be obtained using `GraphDefinitionList Product.Graphs`. Each `GraphDefinition` within that list contains a `GraphId` identifying the graph type, such as frequency response, and can be used to create a graph by calling `Graph GraphDefinition::CreateGraph()`.

To initialize a graph, set the domain of the x-axis by using `Graph::SetDomain(int length, const double* data)`. For a frequency response graph, `data` is an array of sequential frequencies in Hz. For an input/output graph, data is an array of sequential input levels in dB. The size of the data array is specified by `length`.

All graphs contain a list of settings which can be used to specify properties such as the target frequency or signal input level. The settings available per graph vary depending on the graph type. For example, a graph of type Frequency Response has an input level setting, while an Input/Output graph contains a frequency setting. A list of available settings can be obtained from the graph using the `ParameterList Graph::GraphSettings()` method. For more information about the `ParameterList` type, refer to Section 6.5 "Parameter Adjustment" on page 24.

Finally the input signal to the acoustic model needs to be specified. Sound Designer Software supports Pure Tone input signal types.

To calculate the points of a graph use `Graph::CalculatePoints(int length, double* data)`, which returns the y-axis of the graph in the double array `data` of size `length`.

6.8 Transducer Models

To create accurate acoustic models for a product, libraries include a list of transducer models that model the response of various transducers and receivers. These models are created using the Sound Designer Software Modeler tool. These transducer models are used in conjunction with the product model to create an acoustic model that is used for graphing. Methods are available to change the transducer models associated with a given product.

To get a list of transducer models, use the `TransducerModelList Library.TransducerModels` property. You can look up a specific `TransducerModel` from a `TransducerModelList` by specifying the transducer model's string ID and its `TransducerModelType`. The supported types of transducer models are:

- Microphone sensitivity
- Telecoil sensitivity
- DAI sensitivity
- Receiver sensitivity and saturation

Sensitivity is the transfer function from one medium to another. For microphones, it is the voltage output from a sound pressure stimulus. Receivers are the opposite.

Receiver saturation is the maximum output that can be achieved when connected to an onsemi product.

To associate a transducer model with a `Product`, use `Product::SetTransducerModel(TransducerPort port, TransducerModel model)`. The TransducerPort specified corresponds to a transducer model type as shown in the table "Transducer Model Types and Ports" (Table 4). The model data for a transducer can be accessed via the `TransducerModel.Data` property.

Table 4. Transducer Model Types and Ports

TransducerModelType	TransducerPort
kMicrophoneSensitivityModel	kMicrophone1 or kMicrophone2
kTelecoilSensitivityModel	kTelecoil
kDAISensitivityModel	kDAI
kReceiverSaturationModel and kReceiverSensitivityModel	kReceiver1

6.9 Using Voice Alert Data in param Files

Sound Designer Software supports the use of recorded voice clips that can be played back when an event occurs, such as during a volume change or memory selection. Audio data must be provided to the API in G.722 format. Each voice alert data block consists of:

- Header information word (code byte 16-bit) (size of alert 16-bit)
- Hash value of encoded data (32-bit); the sole purpose of this hash is to serve as a unique identifier for the encoded alert data. You can choose the format.
- Encoded alert data

Encoded voice alerts are dealt with in the following manner:

1. When creating a voice alert, Sound Designer Software G.722 encodes the *.wav* file audio data, and constructs the appropriate voice alert header (with codec info word, hash and length information), which is included in memory as a large byte array.
2. When the *.param* file is saved, this is a standard JSON.NET serialization to JSON. When byte arrays are serialized, they are converted to Base64-encoded strings.
3. To read the voice alert data from a *.param* file and download it to the device, do the following:
    a. Parse/de-serialize the JSON file (*.param* file) to objects.
    b. Create a large byte array. For this example, it is named `voicealerts`.
    c. For each Voice Alert, in ascending order of `voicealertindex`:
        i. Base64-decode the `encodeddata` property into a byte array.
        ii. Add this data to the end of the large byte array `voicealerts`.

    d. Call the `manufacturing.WriteVoiceAlert(len(voicealerts), voicealerts)` function.

6.9.1 Write Voice Alerts

To write voice alerts to the device, first sequentially combine all voice alert data blocks into a single contiguous block of data. Then use `Product::WriteVoiceAlert(int length, const unsigned char* data)` to write this data block to the voice alert EEPROM space. It appends the unused space with zeros and writes all the voice alert data to the EEPROM space.

Parameter usage:

length
The amount of total data you want to write

data

Actual data you want to write (combining all alerts sequentially, indexing from 0)

If you want to track the usage of EEPROM space dedicated to voice alerts, you can use the `int Product::ReadVoiceAlertsTotalMemory()` method. It returns a value in bytes. After calling `WriteVoiceAlert` to store the sound files in EEPROM, you can determine how much EEPROM space is used with the `int Product::ReadVoiceAlertsUsedMemory()` method.

6.9.2 Read and Verify Voice Alerts

The header of each voice alert contains a 32-bit hash value generated by applying a CRC32 algorithm on the encoded wave file content for that alert. To verify voice alerts on the device, you can first compute hash values for wave files in the workspace, and then compare them with hash values on the device.

`Product::ReadVoiceAlertHashes(int length, unsigned char* data)` is responsible for retrieving hash values on the device. It returns an array of bytes containing the alert hashes sequentially in a single block of data. The length of data to read back is determined by calling `int Product::GetAlertCount()` which returns the number of voice alerts configured on the device. As noted above, the hash for each alert is 32-bits or 4 bytes; therefore the number of hash bytes to read back is the number of alerts times 4.

6.9.3 Audition Configured Acoustic Indicators

An Acoustic Indicator for a designated event can be played by calling `Product::PlayAcousticIndicator (string eventname)`. Event name options are listed as values of the X_AI_Audition parameter.

6.10 Data Logging

Some products can record usage data over a period of time, and the data can be read back from the device.

6.10.1 Retrieve Data

The method to retrieve these data logs is `const char* Product::GenerateLog (TypeLog log)`, where:

- `log` represents the log type to retrieve. Use `TypeLog` constant `kShort` for the short-term data log, `kLong` for the long-term data log, or `kUsageTime` for the usage time log. Currently, the short-term data log is not accessible at the API level.
- The function returns a JSON format data log string.

NOTE: Before using the `GenerateLog()` method, ensure that `Product` is properly initialized. See Section 6.2.4 "Working with a Product" on page 22 for more information.

The API provides an asynchronous method to retrieve the data log: use `AsyncResult Product::BeginGenerateLog(LogType log)` and `Product::EndGenerateLog(asyncResult)`. See Section 2.7 "Asynchronous Non-Blocking Methods" on page 8 for more information.

6.10.2 Data Log Settings

The following data log settings are available:

Enable/disable

Use parameter `X_DL_Enable`. Set the value to 1 to enable, or 0 to disable.

Time period

Represents the logging frequency and can be changed by using variable `X_DL_TimePeriod`. We provide this set of valid values in seconds: 1, 10, 30, 60, 300, 600, 900, 1200, 1800 and 3600.

NOTE: This time period setting does not affect the frequency of logging in the usage time log, which is set to 10-minute intervals.

Broadband Energy Level Threshold

Use parameter `X_DL_BroadbandEnergyLevelThreshold` to register changes to the front microphone broadband energy levels as a short term log event. If the amount of change to the energy level exceeds this value, the event is logged. Possible values are 0 to 127. Set the value to 127 to disable.

Refer to Section 6.5 "Parameter Adjustment" on page 24 for information about changing parameters.

These settings are common for both the short and long term data logs.

6.10.3 Data Log Reset

The method to reset the data log is `Product::ResetDataLog(LogType log)`. If the type is `kUsageTime`, it resets the usage time log (i.e., sends the command to the counters so that the next time they are read, the resettable counter is 0); if it is one of the other types, it resets the log.

This method resets both the short and long term data logs, and writes 0x00 in the entire log.

6.10.4 Data Log Structure

This section describes the format for each type of data log.

6.10.4.1 Long-Term Data Log

The long-term data log contains records that indicate the time spent in each memory (i.e., memory A, memory B, etc.). Each long-term log record is a 24-bit counter indicating the time based on the time period. The time period is configurable in seconds (minimum 1 second; other valid values are described with the *Time period* definition in Section 6.10.2 "Data Log Settings" on page 28). The long-term log structure is described in the table "Record Format for Long-Term Data Log" (Table 5).

Table 5. Record Format for Long-Term Data Log

Index	Usage
0	Memory A counter
1	Memory B counter
...	...
Number of Memories - 1	Memory (N-1) counter

6.10.4.2 Short-Term Data Log

The short-term data log contains predefined 48-bit records where each record can contain one or multiple parameters as detailed in the table "Record Format for Short-Term Data Log" (Table 6) and the table "Short-Term Data Log Record Types" (Table 7).

In short-term data logging, data is periodically recorded in a circular buffer. The parameters being logged are immediately recorded at device power up, and checked at regular time intervals determined by the time period setting.

When the parameters are checked, data is only recorded if one of the following occurs:

- A parameter value changes.
- The timestamp value is at its maximum and is about to wrap around. Parameters must be recorded when the timestamp value is about to wrap around to properly calculate the actual time that the hearing aid has been on.

Each extracted data log record contains a `Time` field representing the time in seconds since the device was on. The `Time` field is calculated using:

- The `Time Stamp`
- The time period setting
- The count found in a special record called `Time Wrap`

Table 6. Record Format for Short-Term Data Log

Bit Field	Field Name	Description
47:40	Record Identifier	Identifier that describes the record type
39:32	Time Stamp	Number of data logging periods elapsed since power on
31:0	Data	Data value for this record

Table 7. Short-Term Data Log Record Types

Record Identifier	Field Name	Description
0	Reserved	Reserved
1	Time Wrap	A count of the number of times the time stamp has wrapped back to 0.
2	System Log	Contains general system information, including current battery voltage, volume level and selected memory
3	Long-term total	Current sum of the long-term counters
3-255	Not used	Not used

The table "System Log Record" (Table 8) provides the content of the System Log record.

Table 8. System Log Record

Bit Field	Field Name	Description
31:25	Volume	The volume level in dBFS
24:15	Battery level	The battery level in Volts
14:11	Current Memory	The current selected memory
10:7	Environment Classifier	Environment Classifier type
6:0	Front Microphone	The broadband energy level of the front microphone in dB10

The table "Long-Term Total Record" (Table 9) provides the content of the Long-Term Total Record.

Table 9. Long-Term Total Record

Bit Field	Field Name	Description
31:0	Long-term total	Unsigned integer value representing the sum of the long-term counters at power-on. To convert this to a time value, multiply by the data logging period.

Example of an extracted short term data log record in JSON format string:

```
{
  "Short Term Data Log": [
    {
      "Battery level": "1.28125",
      "Environment": "Unclassified",
      "Memory": "A",
      "Mic Broadband Energy Level": "-83",
      "Raw data": "0x02050CA407D3",
      "Record id": "2",
      "Record type": "System log",
      "Time": "5",
      "Volume": "-6"
    },
   ]
}
```

6.10.4.3 Usage Time Data Log

The usage time data log contains the number of times the counters have incremented (i.e., the number of 10-minute intervals that have passed), which indicates how long the device has been used. The record consists of two fields: the first 24 bits are the resettable counter and the next 24 bits are the non-resettable counter.

An example of an extracted usage time data log record in JSON format is:

```
{
  "Usage Time Data Log": [
    {
      "Resettable Counter": "28"
    },
    {
      "Non-Resettable Counter": "28"
    }
  ]
}
```

6.10.5 Data Log JSON File Structure

The data log is saved in a JSON file format with the following structure:

```
{
  "Long Term Data Log": [
    {
      "Memory": A,
      "Time": "125"
    },
    {...},
    {
      "Memory": X,
      "Time": "234"
    }
```

```
  ]
}
```

Where:

- "Time" represents the time in seconds spent in the memory.
- "Memory" represents the selected memory (active memory).

6.11 Other Uses for the Product Class

6.11.1 Muting and Unmuting

The audio path of the connected device can be temporarily muted using `Product:MuteDevice(bool enable)`. All parameters that affect the audio path are unchanged while the device is muted.

6.11.2 Reading and Writing the Manufacturing Area

Some devices can contain an area of NVM dedicated to manufacturing data. This area was called the *scratch space* in ARK, the predecessor of Sound Designer Software. This area is not read or modified by the device. It can be used to store any information that the manufacturer might require. The number of bytes of storage allocated for this data can vary according to the product. The property `int ProductDefinition.ManufacturerDataAreaLength` returns the size of the manufacturing data area in bytes.

To read this data, use the `Product::ReadManufacturerData(int offset, int length, unsigned char* data)` method. The data read back is stored in the byte array `data` of size `length`. The `offset` argument can be used to start reading from an area other than the start of the manufacturing data area.

To write this data, use the `Product::WriteManufacturerData(int offset, int length, const unsigned char* data)` method. The data to be written needs to be stored in the byte array `data` of size `length`. The `offset` argument can be used to start writing to an area other than the start of the manufacturing data area.

The `offset` and `length` need to be in multiples of three because data is read and written from the device in a minimum of 24-bit words.

In addition to the above methods, the API provides methods to read and write manufacturing data asynchronously by using `Product::EndReadManufacturerData(AsyncResult async, int length, unsigned char* data)` and `AsyncResult Product::BeginWriteManufacturerData(int offset, int length, const unsigned char* data)`. See Section 2.7 "Asynchronous Non-Blocking Methods" on page 8 for more information.

6.11.3 Automatic Receiver Detection (ARD)

Some devices support the Automatic Receiver Detection feature. Each receiver in a product is assigned a unique identifier with a value 0-8. This ID can be read out from the device using the `Product::ReadArdId()` method which returns the ID associated with the currently connected receiver.

6.11.4 Feedback Path Measurement Tool

Some devices have a Feedback Path Measurement Tool module for measuring the device's feedback path. To run the tool, use the asynchronous method `AsyncResult Product::BeginRunChirpTool()`. Note that this method will cause all system parameters to be written to System Active Memory. See Section 2.7 "Asynchronous Non-Blocking Methods" on page 8 for more information on asynchronous methods.

Once the asynchronous method has finished, the Feedback Path Measurement Tool results can be read from the properties `ChirpToolResultList Product.ChirpToolFrequencyResponse` and `ChirpToolResultList Product.ChirpToolImpulseResponse`. The `ChirpToolFrequencyResponse` variable contains the measured amplitude of the feedback path at linearly spaced frequencies from zero to half of the sampling frequency. The `ChirpToolImpulseResponse` variable contains the measured impulse response of the feedback path at the device sampling frequency. Both results are lists of type double.

6.12 Finish Working with a Device

When you have finished working with a product and device, close the connection between the two. The method `Product::CloseDevice()` is used to do this. This ensures that the device is ready to be used, and the connection to it via the `CommunicationAdaptor` is closed.

CHAPTER 7

Live Display Interface

The purpose of live displays are to provide access to real-time data from the device to the user.

7.1 Live Display Properties

Each live display has the following properties:

Name
The name of the current live display object.

Type
The enumerated type of the current live display which corresponds to the module of interest.

Use the following method to extract the *Name* of the live display:

```
const char* LiveDisplay::GetName()
```

To get the *Type* of the live display, call the following method:

```
LiveDisplayType LiveDisplay::GetType()
```

Where `LiveDisplay` is the current display object of type `LiveDisplay`. Possible types are described in the table "Types of Live Displays Available" (Table 10).

Table 10. Types of Live Displays Available

Live Display Type	Description
kAGCO	Automatic gain controller output level detector and gain
kFeedbackCanceller	Feedback canceller level detector and amplitude
kWDRC	Wideband dynamic range compressor level detectors
kNoiseReduction	Noise reduction SNR and frequency gains

7.2 Creating A Live Display

To create a live display, extract a list from the product using the following property:

```
LiveDisplayList Product.GetLiveDisplays
```

Select the live display by specifying the display type from the table "Types of Live Displays Available" (Table 10). This is done using the following method:

```
LiveDisplay LiveDisplayList::GetById(LiveDisplayType type)
```

7.3 Starting a Live Display

Each live display must be enabled before real-time data can be read from it. Failing to do so results in an `E_INVALID_STATE` error. A live display can be read multiple times after it has been enabled. To enable the live display, call the following method:

```
void LiveDisplay::Start()
```

7.4 Reading a Live Display

Live display data is read from the device and formatted to a JSON string. All live display output strings have the following format: `{"Display Value Name": Value}`. The following method reads data from the live display:

```
const char* LiveDisplay::Read()
```

7.5 Stopping a Live Display

Disable the live display module after the data has been read. Disabling the module allows other modules to be enabled and read without undefined behavior. This is done by calling the following method:

```
void LiveDisplay::Stop()
```

CHAPTER 8

Working with a Wireless Device

If you are working with a hearing aid that has wireless capability, this chapter explains how to use the SDK to access those features.

8.1 Scanning for and Connecting to a Device

The SDK provides a set of methods in the `ProductManager` which can be used to scan for and connect to compatible hearing aid devices. These methods are applicable on all supported platforms.

On iOS, you can get a list of connected devices using the `iOSConnectedDeviceList ProductManager::iOSConnectedDevices` method. This list is used to identify devices that are already connected to the phone via MFi. The ID from a member of the return list can be used with the `ProductManager::CreateWirelessCommunicationInterface(const char* uuid)` method described below.

To get a list of devices that are not yet connected, use the method `AsyncResult ProductManager::BeginScanForWirelessDevices(WirelessProgrammerType interface_name, const char* com_port, CommunicationPort port, const char* interface_options, bool clear_bond_table)` to initiate the scanning process. On mobile platforms, set `interface_name` to `kPlatformDefault` while leaving `com_port` and `interface_options` as empty strings. On Windows, the currently supported value for `interface_name` is `kNoahlinkWireless` and `kRSL10`; enter `com_port` for `kRSL10`, not `kNoahlinkWireless`, and `interface_options` is an empty string for both. Set the port input argument to either `CommunicationPort::kLeft` or `CommunicationPort::kRight`. Leave `clear_bond_table` set to false. Since this is an asynchronous method, see Section 2.7 “Asynchronous Non-Blocking Methods” on page 8 for more information.

After the scanning process has initiated, the SDK generates an `Event` of type `EventType::kScanEvent` for each compatible device detected. See Section 8.2 “Handling Events from the SDK” on page 36 to work with the `EventHandler` class. Each returned Event contains a device name and ID in the JSON `Data` section. The SDK uses the device ID as the unique identifier for a scanned device.

Once all nearby devices have been scanned, the SDK can be instructed to end scanning with the `ScanList EndScanForWirelessDevices(AsyncResult async)`.

With the scanned device ID, use the `ProductManager::CreateWirelessCommunicationInterface(const char* uuid)` method to get a `CommunicationAdaptor` class that will be used for device communication. Call `CommunicationAdaptor::Connect()` to initiate the connection process. The SDK generates an `Event` of type `EventType::kConnectionEvent` with the JSON `Data` section specifying the current connection state. Once connected, the `CommunicationAdaptor` behaves as described in Section 5.3 “Using the Communication Adaptor Class” on page 20.

To disconnect from a device, first call `CommunicationAdaptor::Close()` and then `CommunicationAdaptor::Disconnect()`. The SDK generates an `Event` of type `EventType::kConnectionEvent` with the JSON `Data` section specifying the disconnection state.

8.2 Handling Events from the SDK

The `EventHandler` interface provides a way for the SDK to return asynchronous events. Events can include scan results for nearby detected Bluetooth devices or notifications from a given Bluetooth characteristic when a change occurs. If a new `Event` is present in the SDK calling `Event EventHandler::GetEvent()` returns the event. Events are buffered in a queue within the SDK until they are checked for using `GetEvent()`. If the internal event queue is empty and a call to `GetEvent()` is made, this method blocks until a new event arrives. For this reason, we recommend

calling this method in a loop on a separate thread. If there are no new events expected, the blocking `GetEvent()` method can be escaped by calling `EventHandler::Exit()`. This will fire an Event with type `kExitEvent`. This method should only be called once the `EventHandler` is no longer needed.

Each `Event` class contains two properties: `Type` and `Data`. `Type` is an `EventType` enumeration and `Data` is a string in JSON format. the table "Event Types and Data Formats" (Table 11) gives a list of the types of events and example data formats. Additionally, a list of supported types of events can be found in the *Ezairo Sound Designer Application Programming Interface Reference*.

Table 11. Event Types and Data Formats

Event Types	Example Data Format
kVolumeEvent	{ "Event" : [{ "DeviceID" : "40496EDB-AA55-B14E-C898-1F1B92713AFF" }, { "VolumeLevel" : "88" }] }
kScanEvent	{ "Event" : [{ "DeviceName" : "R61" }, { "DeviceID" : "40496EDB-AA55-B14E-C898-1F1B92713AFF" }, { "RSSI" : "-54" }, { "ManufacturingData" : "6203000000" }] }
kConnectionEvent	{ "Event" : [{ "DeviceID" : "40496EDB-AA55-B14E-C898-1F1B92713AFF" }, { "ConnectionState" : "3" }, { "ErrorDesc" : "" }] }
kActiveEvent	{ "Event" : [{ "IsActive" : "1" }] }
kMemoryEvent	{ "Event" : [{ "DeviceID" : "40496EDB-AA55-B14E-C898-1F1B92713AFF" }, { "CurrentMemory" : "1" }] }
kBatteryEvent	{ "Event" : [{ "DeviceID" : "40496EDB-AA55-B14E-C898-1F1B92713AFF" }, { "BatteryLevel" : "63" }] }
kAuxAttenuationEvent	{ "Event" : [{ "DeviceID" : "40496EDB-AA55-B14E-C898-1F1B92713AFF" }, { "AuxLevel" : "94" }] }
kMicAttenuationEvent	{ "Event" : [{ "DeviceID" : "40496EDB-AA55-B14E-C898-1F1B92713AFF" }, { "MicLevel" : "82" }] }
kExitEvent	{ "Event" : [ ]

8.3 Handling Disconnects

When connected to a hearing aid device over a wireless connection, unexpected disconnects may sometimes occur. When this happens, the SDK generates an `Event` of type `EventType::kConnectionEvent` with the JSON `Data` section specifying the disconnection state. Any attempt at communicating with the hearing aid device at this point returns an error. A connection with the hearing aid must be re-established before proceeding.

In most cases, just waiting 2-5 seconds, calling `CommunicationAdaptor::Connect()` again, and then waiting for the connected event is sufficient to continue working with the hearing aid. But in some situations a new `CommunicationAdaptor` might be required. If the scanned device ID is still known, it can be used to create a new `CommunicationAdaptor` using `ProductManager::CreateWirelessCommunicationInterface(const char* uuid)` as described in Section 8.1 "Scanning for and Connecting to a Device" on page 36. Next, call `CommunicationAdaptor::DetectDevice()` to confirm the identity of the device. Upon successfully detecting the device, this new `CommunicationAdaptor` needs to be provided to the existing `Product` instance so that it can be used to communicate. Do this by calling `Product::InitializeDevice(CommunicationAdaptor adaptor)` with the new `CommunicationAdaptor`.

To minimize the communication impact that an unexpected disconnection causes, during the `Product::InitializeDevice(CommunicationAdaptor adaptor)` call, if the SDK recognizes that it is communicating with the same the hearing aid device as it was prior to the disconnection, it avoids unnecessary communication steps intelligently, which reduces the time needed to initialize. Once initialized, the `Product` can be used again.

8.4 Controlling a Device via Bluetooth Characteristics

Some devices support Bluetooth low energy services and characteristics which can be used to interact with the device. The SDK provides the `WirelessControl` interface as a way to use these. This interface exposes properties which are used to view and change the current value of a given characteristic such as the volume level. A complete list of the characteristics can be found in the *Ezairo Sound Designer Application Programming Interface Reference* section for `WirelessControl`.

The first step in using the `WirelessControl` interface is to pass in a `CommunicationAdaptor` with an established connection via `WirelessControl::SetCommunicationAdaptor(CommunicationAdaptor adaptor)` method. In order to get the current volume level the `WirelessControl.Volume` property can be used. Setting a value to this property will change its value on the device.

Some Bluetooth characteristics support notifications from the device when a change occurs. For example if the volume is changed on the device a notification will be triggered. These notifications will arrive in the SDK as an `Event` in the `EventHandler`. Details on the `EventHandler` can be found in Section 8.2 "Handling Events from the SDK" on page 36.

8.5 Updating MFi Information on a Device

MFi information on supported devices can be updated using the SDK. The `UpdateInfoOnDevice` method is included in SDK. To upload the information to the device, it requires a DCR certificate file and certificate number from the device manufacturer.

An Asynchronous version of the method, `BeginUpdateMFiInfoOnDevice` , can be run in the background and provide progress information.

With both methods, the device reboots a couple of times and the operation is expected to take at least 7-8 seconds.

8.6 Updating MFi Memory Names and Categories

MFi memory names and categories can be updated using two public parameters.

To update MFi memory categories, update the parameters for each memory that needs the updated category:

```
X_RF_MFiProgramCategory[0],X_RF_MFiProgramCategory[1].....X_RF_MFiProgramCategory[7]
```

To update the memory names, the following public parameters are available:

```
X_RF_MFiProgramNames[0]........X_RF_MFiProgramNames[75]
```

The data to be written for Program names has to be converted from string to bytes using code packing based on Ezairo architecture. The packing algorithm is demonstrated in C# below:

```
int MAX_CHAR_LEN = 28
;
string[] names = ["Memory A", "Memory B", "Memory C", "Memory D", "Memory E", "Memory F", "Memory G", "Memory H"]
;
uint[] packed_names = new uint[(Constants.MFI_MAX_NAME_LENGTH * names.Length + 2) / 3];
        for (int i = 0; i < names.Length; i++)
        {
          List<byte> str_byte_arr = Encoding.UTF8.GetBytes(names[i]).ToList();
          str_byte_arr.Add(0);
          for (int j = 0; j < str_byte_arr.Count; j++)
          {
            var word_idx = (Constants.MFI_MAX_NAME_LENGTH * i + j);
            if (word_idx % 3 == 0)
              packed_names[word_idx / 3] = (uint)str_byte_arr[j] << 16;
            else if (word_idx % 3 == 1)
```

```
        packed_names[word_idx / 3] = (packed_names[word_idx / 3]) | (uint)(str_byte_arr[j] << 8);
      else
        packed_names[word_idx / 3] = packed_names[word_idx / 3] | str_byte_arr[j];
    }
}
for (int i = 0; i < packed_names.Length; i++)
{
    var parameter = system.Find(x => x.Name == "X_RF_MFiProgramNames[" + i + "]");
    parameter?.SetValue(packed_names[i].ToString());
}
```

CHAPTER 9

Manufacturing Interface

The Manufacturing Interface contains all the necessary methods and properties to allow you to model and calibrate a device. It provides access to a list of supported acoustic test boxes.

9.1 Acoustic Test Box

For the purposes of modeling and calibration, Sound Designer Software provides a `TestBox` interface that is used to both measure and generate tones using an acoustic test box. The API comes with an implementation for the FONIX 7000/8000 and Audio Precision APx511; however, it can be extended by the user to support other test boxes.

To get a list of supported test boxes, the `Manufacturing.TestBoxList` property of type `TestBoxList` is used. Once a test box is selected, use the `TestBox.Settings` property to get a list of parameters that are used to set up the test box. The `Settings` property is of type `ParameterList`. The `Parameter` objects in the settings list are adjusted in the same way as documented in Section 6.5 “Parameter Adjustment” on page 24. Once the settings for a test box are configured, call `TestBox::Open()`, which establishes a connection to the test box and initializes it. `bool TestBox::IsOpen()` can be used to determine whether this connection is active.

The `TestBox` interface supports the generation and measurement of pure tones and complex signals. To generate a pure tone, use `TestBox::GeneratePureTone(double inputlevel, double freq)`. The input level is specified in dB and the frequency in Hz. To stop playing a tone, use `TestBox::Stop()`. To measure a pure tone, use `FreqPoint TestBox::MeasurePureTone(double freq)`. `FreqPoint` is a structure that contains a frequency, magnitude, and phase measurement.

The complex signal generated by a test box is specific to the implementation of the `TestBox` interface for a given test box. In the case of FONIX-based equipment, the complex signal generated is the Frye Composite tone. (For more information about the Frye Composite tone, refer to the Frye Fonix User’s Manual.) Note that for a composite signal, setting its level is actually setting its overall (RMS) energy, not the amplitude of the individual frequency components. A complex signal is generated using `TestBox::GenerateComplexSignal(double inputlevel)` where `inputlevel` is specified in dB. To measure a complex signal, use `FreqPointList TestBox::MeasureComplexSignal(int length, double* freqs)`, where:

`freqs`
: The list of frequencies at which to perform the measurement

`length`
: The number of frequencies in the list

`FreqPointList`
: A list of measured spectrum levels at the specified frequency points

For modeling or calibrating a telecoil, a magnetic field is used in place of an acoustic signal when generating and performing measurements. To enable this mode on the test box, use the `TestBox::SetMagneticFieldMode(bool generate)` method. If `generate` is True, the magnetic field generator is enabled.

NOTE: The APx511 does not support generating complex tones, measuring complex tones, or the magnetic field generator for telecoil modeling.

Once you are done working with a test box, close the connection; use the `TestBox::Close()` method.

9.2 Modeling

The Manufacturing Interface provides several methods to assist with modeling the transducer response for a hearing aid device. It allows the user to set up a device for modeling, read live level detector data, read preamp gains, and generate tones on the device. This is how the transducer models discussed in Section 6.8 "Transducer Models" on page 26 are generated.

To enable the Manufacturing Interface to communicate with a device, it must be provided a `CommunicationAdaptor` instance. Use the `Manufacturing::SetCommunicationAdaptor (CommunicationAdaptor adaptor)` method. The `CommunicationAdaptor` class is discussed in Chapter 5 "Communication Interfaces" on page 20.

The first step when modeling a device is to put the device into the appropriate state. The method `Manufacturing::PrepareDeviceForModeling(bool enable)` sets up the device audio path, gains and level detectors in the necessary state for modeling.

NOTE: Calling this method is a prerequisite before accessing any of the other modeling methods.

After the device has been prepared for modeling, you can read out level detector values for each input that has a transducer connected. Use the `double Manufacturing::ReadLevelDetector(TransducerPort port)` which returns the level detector value in dBFS. You can also read the current preamp gain value for a given input port using `double Manufacturing::ReadPreampGain(TransducerPort port)`, which returns the gain value in dB.

For the purposes of receiver modeling, the device can be made to generate a tone at a specified frequency and level. The method `Manufacturing::GenerateTone(int frequency, int volume)` generates a tone where `frequency` is in Hz, and `volume` is in dB. Use `Manufacturing::StopTone()` to stop the tone.

9.3 Calibration

The first step in calibration is to provide the Manufacturing Interface with the product instance to be calibrated. Use the method `Manufacturing::SetProduct(Product product)`. The `Product` class is documented in Section 6.2.4 "Working with a Product" on page 22.

To set the compensation gains in the product, use the method `Manufacturing::SetCompensationGain (TransducerPort port, double gain)`. After a product is configured, the gains are stored to the device.

9.4 Configure

The `Configure` method commits all parameter values in all memories to the device. This also includes the compensation gains, so calibration must be performed before this method is called.

If required, a serial number for a product can be set using `Manufacturing::SetSerialID(int serial)`. It is written to the device after the product is configured.

A device can optionally contain a parameter lock key which is used to restrict read and write access to all parameters. This key consists of four 24-bit words and can be set using `Manufacturing::SetParameterLockKey (int key0, int key1, int key2, int key3)`. If you want to remove an existing key, set each 24-bit key to 0. The key is written to the device after the product is configured.

Once a device has been calibrated, it can be configured using the method `Manufacturing::Configure()`. This writes all the parameter values stored in the product, including the compensation gain values and calibration settings, to the device for all memories.

onsemi and the onsemi logo are registered trademarks of Semiconductor Components Industries, LLC (onsemi). All other brand names and product names appearing in this document are trademarks of their respective holders.

onsemi and the onsemi logo are trademarks of Semiconductor Components Industries, LLC dba onsemi or its subsidiaries in the United States and/or other countries. onsemi owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of onsemi's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. onsemi reserves the right to make changes without further notice to any products herein. onsemi makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does onsemi assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. Buyer is responsible for its products and applications using onsemi products, including compliance with all laws, regulations and safety requirements or standards, regardless of any support or applications information provided by onsemi. "Typical" parameters which may be provided in onsemi data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. onsemi does not convey any license under its patent rights nor the rights of others. onsemi products are not designed, intended, or authorized for use as a critical component in life support systems or any FDA Class 3 medical devices or medical devices with a same or similar classification in a foreign jurisdiction or any devices intended for implantation in the human body. Should Buyer purchase or use onsemi products for any such unintended or unauthorized application, Buyer shall indemnify and hold onsemi and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that onsemi was negligent regarding the design or manufacture of the part. onsemi is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:

Literature Distribution Center for onsemi

19521 E. 32nd Pkwy, Aurora, Colorado 80011 USA

Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada

Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada

Email: orderlit@onsemi.com

N. American Technical Support:

800-282-9855 Toll Free USA/Canada

Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910

onsemi Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

M-20815-031